CPS 3320 Python Programming
Project 7

Restaurant Project Description

In this project, you will create a database driven restaurant menu ordering system. The menu will include options for selecting:

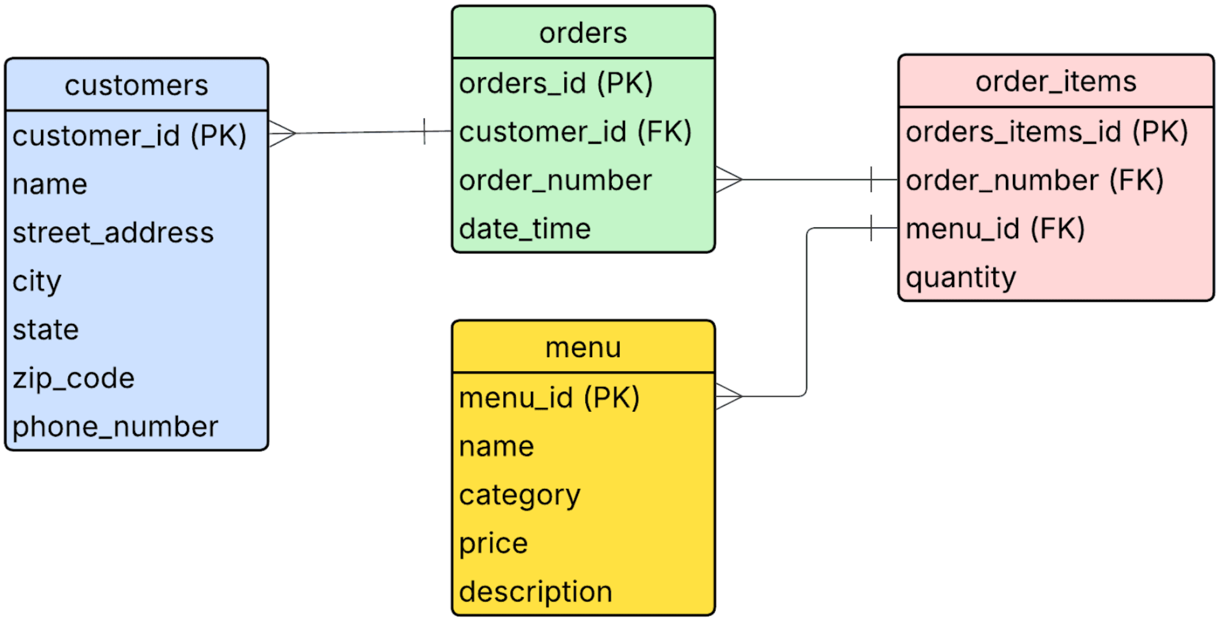
- Restaurant Information
- Admin Menu
- Start New Order

Starter Code

$ code project7.py

[Source Code]

Tables



menu table

After creating this table, insert the menu items per your previous restaurant project.

Column	Type	Description
menu_id	INTEGER	Primary key (unique item ID) AUTOINCREMENT
name	TEXT	Name of the item
category	TEXT	appetizers, entrees, desserts, beverages
price	REAL	Price of the item
description	TEXT	Provide a description for the customer

customers table

After creating this table, insert a customer for testing purposes.

Column	Type	Description

customer_id	INTEGER	Primary key (unique item ID) AUTOINCREMENT
name	TEXT	Customer Name
street_address	TEXT	Customer Address
city	TEXT	Customer city
state	TEXT	Customer state
zip_code	TEXT	Customer zip code
phone_number	TEXT	Customer phone number (201) 000-0000

orders table

The orders table will store the order_number associated with the customer_id and date_time.

Column	Type	Description
order_id	INTEGER	Primary key (unique item ID) AUTOINCREMENT
customer_id	INTEGER	Foreign key → customers.customer_id
order_number	TEXT	Generate a unique string e.g. letters with an incrementing number i.e. ABC-20250410122401 ABC followed by the timestamp values YYYYMMDDHHMMSS
date_time	TEXT	Timestamp of order YYYY-MM-DD HH:MM:SS

order_items table

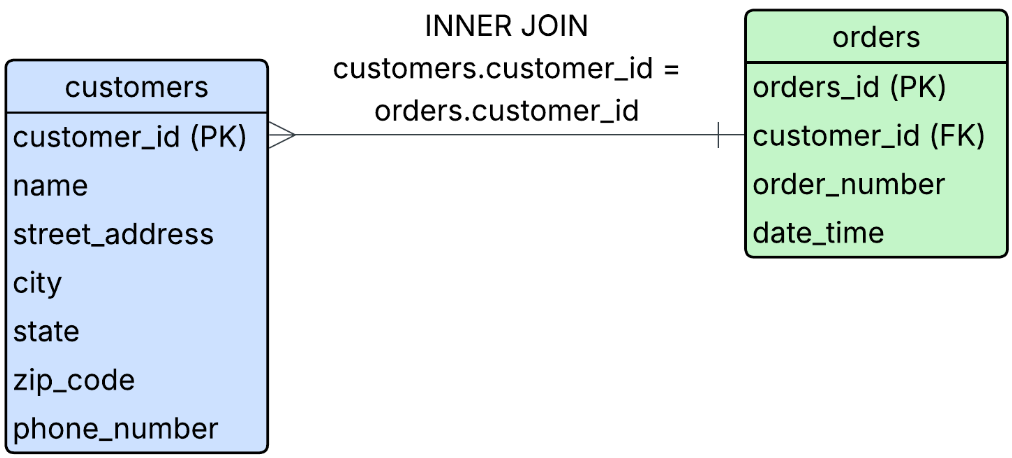
Each row = one item in a specific order.

Column	Type	Description

order_item_id	INTEGER	Primary key (unique item ID) AUTOINCREMENT
order_number	INTEGER	Foreign key → orders.order_number
menu_id	INTEGER	Foreign key → menu.menu_id
quantity	INTEGER	Quantity ordered

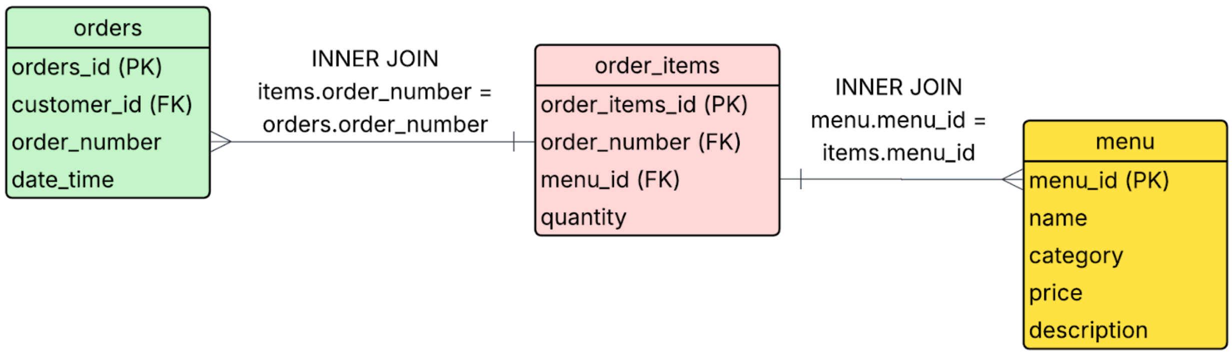
INNER JOINS

Display a customer's orders.



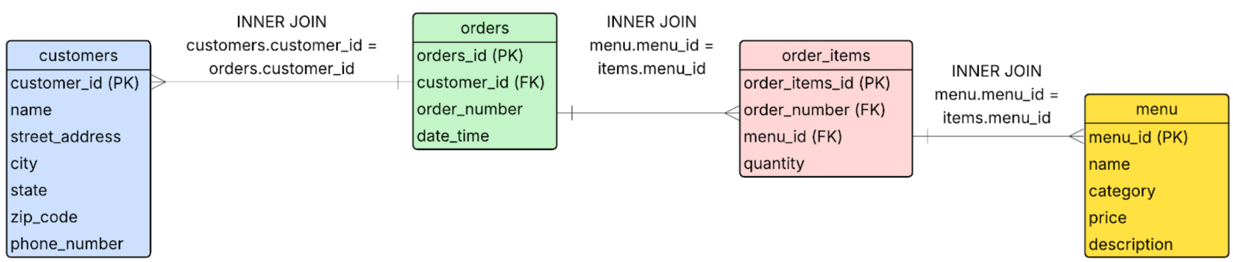

```
SELECT customers.customer_id, customers.name, orders.order_number, orders.date_time
FROM customers
INNER JOIN orders ON customers.customer_id = orders.customer_id
WHERE …
```

Display the items in an order.




```
SELECT order_items.quantity, orders.order_number, menu.menu_id, menu.name, menu.price
FROM orders
INNER JOIN order_items ON order_items.order_number = orders.order_number
INNER JOIN menu ON menu.menu_id = order_items.menu_id
WHERE …
```

Display a customer's orders and the items in the order



Example Workflow

1. Ask the user if they are a new or existing customer.
2. Create a unique order_id.
3. Let's say a customer places an order for:
    - 1x Calamari
    - 2x Cheeseburgers
    - 1x Fries
    - 1x Soda
    - 3x Chocolate Cake slices

Then your tables would store:

- 1 row in customers table (if they are a new customer)
- 1 row in orders with customer_id, order_number and date time

- 5 rows in order_items, each with the <u>same</u> order_id but <u>different</u> menu_id and quantities

Specifications

main function

- [ ] Admin Menu
    - [ ] Create menu table
    - [ ] Create orders table
    - [ ] Create order_items table
    - [ ] Create customers table
    - [ ] View all menu items
    - [ ] View all customers table
    - [ ] View past orders
        - [ ] Show all past order numbers
        - [ ] Ask the user for the order number
            - [ ] Display the items in the order number
    - [ ] Insert menu items
- [ ] Start New Order
    - [ ] Description: The restaurant ordering menu will display the following based on the item's category.
    - [ ] Ask the user if they are a new or existing customer.
        - [ ] If they are a new customer, create a new customer.
            - [ ] Ask the user for their name, address, state, city, zip phone number
            - [ ] Insert into the customer table
        - [ ] If they are an existing customer:
            - [ ] Ask them for their customer_id (display the current customers so they can see their id) or
            - [ ] Search by the customer's name.
    - [ ] Create a new order_number (a unique string) with customer_id and date_time
    - [ ] Insert the customer_id, order_number and date_time into orders
    - [ ] For example, when you place an order at a store, there is a unique order number.

Order Number: ABC-123

Order Place: 2026-04-15 19:92:23 00:00:00

- [ ] Ask the user what category they'd like to order.
    - [ ] Appetizers
    - [ ] Entrees
    - [ ] Desserts
    - [ ] Beverages
    - [ ] Display the items on the menu with a brief description and allow the customer to order.
    - [ ] Prompt the user to enter the ID of the menu item they wish to order and the quantity.
    - [ ] Insert the order_id, menu_id, quantity into order_items
    - [ ] Think about the scenario when someone tries to order an item that they've already ordered. Should you add a new row or change the quantity?
    - [ ] Allow the user to continue ordering from the menu until they are finished.
- [ ] Modify Order
    - [ ] Display the current order.
    - [ ] Prompt the user to enter the ID of the item they wish to update/delete.
    - [ ] Modify accordingly.
- [ ] Place Order
    - [ ] Display all the items in the order in a readable format, including the name, quantity, price, and order time.
    - [ ] Add them together to print a subtotal.
    - [ ] Ask the user if they have a coupon.
        - [ ] Ask the user if the coupon is a % off or a specific amount off.
        - [ ] Subtract the appropriate amount.
    - [ ] Ask the user if they'd like to tip 15%, 18% or 20%.
    - [ ] Add New Jersey sales tax of 6.625%.
    - [ ] Ask the user if they'd like pickup or delivery.
        - [ ] If the user chooses delivery, add $5 to their order.
    - [ ] Print the total of the order.

Notes

- To reset the AUTOINCREMENT, use this command:
    - DELETE from sqlite_sequence where name='table_name';

Project Rubric

Description	Points

1. Create menu, customers, orders and order_items table	10
2. Created Admin Functionality	30
3. Restaurant Ordering System	40
4. Screen Recording including narration • Demonstrate ordering from each menu choice • Demonstrate Admin Functionality • Demonstrate Placing an Order • Demonstrate Customer Information and their orders	20
Total	100